

目次

1	プロジェクト概要	リポジトリの目的・基本情報
2	ディレクトリ構成	全ファイル・フォルダの配置
3	設定ファイル (settings.json)	権限ルール・フック設定
4	エージェント定義	security-reviewer エージェント
5	カスタムコマンド	/deploy コマンドの手順
6	スキル定義	code-review / testing スキル
7	スクリプト	Windows 自動同期スクリプト
8	ワークフロールール	CLAUDE.md に定義された開発ルール
9	Git 情報	ブランチ・コミット履歴

1 プロジェクト概要

リポジトリ名	oaku-bit/claude
目的・用途	Claude Code の自動化・運用ワークスペース。 設定・スキル・エージェント・スクリプトを一元管理する。
現在のブランチ	claude/quirky-newton-vLU0Z（開発用）
ベースブランチ	master
リモート URL	GitHub（ローカルプロキシ 127.0.0.1:32817 経由）
リポジトリ種別	Git リポジトリ（クラウド実行環境にクローン済み）
作成日	2026年5月22日

2 ディレクトリ構成

ファイルツリー（.git ディレクトリを除く全ファイル）

```
/home/user/claude/
├── CLAUDE.md ← プロジェクトルール・ガイドライン（最重要）
├── README.md ← リポジトリ概要（1行）
├── .gitignore ← Git 除外設定
├── claude_code_environment_report.pdf ← 本レポート PDF
├── generate_report.py ← PDF 生成スクリプト（Python）
├── scripts/
│   ├── auto-sync.ps1 ← Windows 自動同期 PowerShell スクリプト
│   └── setup-task-scheduler.ps1 ← Windows タスクスケジューラ登録スクリプト
├── .claude/ ← Claude Code 設定ディレクトリ
├── settings.json ← 権限・フック設定（gitignore 対象外・追跡済み）
├── agents/
│   └── security-reviewer.md ← セキュリティレビュー専用エージェント定義
├── commands/
│   └── deploy.md ← /deploy スラッシュコマンド定義
├── skills/
│   ├── code-review/SKILL.md ← コードレビュースキル定義
│   └── testing/SKILL.md ← テストスキル定義
└── hooks/ ← フックスクリプト置き場（.gitignore 除外）
```

ファイル種別サマリー

ファイル種別	件数	内容
Markdown (.md)	5 ファイル	CLAUDE.md / README.md / agents / commands / skills
JSON (.json)	1 ファイル	.claude/settings.json (権限・フック設定)
PowerShell (.ps1)	2 ファイル	auto-sync.ps1 / setup-task-scheduler.ps1
Python (.py)	1 ファイル	generate_report.py (PDF 生成)
PDF (.pdf)	1 ファイル	claude_code_environment_report.pdf

3 設定ファイル (.claude/settings.json)

3-1. 許可ルール (permissions.allow)

Read:*	Bash:git:*	Write:*	Edit:*
Read:*	ワークスペース内の全ファイルの読み取りを許可		
Bash:git:*	git コマンドのみ Bash 実行を許可 (他のコマンドは個別許可が必要)		
Write:*	全ファイルへの書き込みを許可		
Edit:*	全ファイルの編集 (差分更新) を許可		

3-2. 拒否ルール (permissions.deny)

Read:.env* (.env 読み取り禁止)	Bash:sudo:* (sudo 禁止)	Bash:rm -rf / (ルート削除禁止)
Read:.env*	環境変数ファイル (.env / .env.local など) の読み取りを強制禁止	
Bash:sudo:*	管理者権限コマンドの実行を全面禁止	
Bash:rm -rf /	ルートディレクトリ以下の再帰削除コマンドを禁止 (誤操作防止)	

3-3. フック設定 (hooks)

トリガータイミング	Stop — Claude セッションが停止するたびに実行
-----------	-------------------------------

フックタイプ	command（シェルコマンドを直接実行）
実行コマンド	bash /home/user/claude/.claude/hooks/auto-push.sh
タイムアウト	30 秒（超過した場合は強制終了）
目的・効果	作業内容をセッション終了ごとに自動でリモートへ push し、クラウド環境のデータ消失を防ぐ

4 エージェント定義（.claude/agents/）

ファイル	.claude/agents/security-reviewer.md
エージェント名	security-reviewer
説明	OWASP Top 10 を基準に、コード変更のセキュリティ脆弱性を検出・報告する専門エージェント
起動方法	/review コマンドや手動指定で呼び出す

検査項目（OWASP Top 10 準拠）

No	カテゴリ	検査内容
1	Injection	SQL インジェクション / コマンドインジェクション / XSS
2	認証・セッション管理	ハードコードされた認証情報 / 弱いトークン / セッション固定
3	機密データ露出	.env ファイル / API キー / パスワードの平文保存
4	アクセス制御	権限チェック漏れ / 水平権限昇格 / 直接オブジェクト参照
5	セキュリティ設定ミス	デフォルト設定のまま運用 / 不要な機能の有効化

出力フォーマット

セキュリティレビュー結果

CRITICAL:（重大な脆弱性 — 即時対応必須）

HIGH:（高リスク — 早急な対応が必要）

MEDIUM:（中リスク — 計画的に対応）

LOW:（低リスク — 余裕があるときに対応）

5 カスタムコマンド（.claude/commands/）

ファイル	.claude/commands/deploy.md
------	----------------------------

コマンド名	/deploy
目的	デプロイ前のチェックリストを順番に実行し、安全にリモートへリリースする

/deploy コマンド — 実行ステップ詳細

No	手順	内容	コマンド例
1	テスト実行	npm test などで全テストがパスすることを確認	npm test
2	Lint チェック	コード品質と構文エラーをチェック	npm run lint
3	機密情報スキャン	SECRET / PASSWORD / API_KEY が含まれていないか検査	git diff --name-only xargs grep -l 'SECRET API_KEY'
4	ブランチ確認	main または release/* ブランチ上にいることを確認	git branch --show-current
5	差分最終確認	git diff main で変更内容を人の目で最終チェック	git diff main
6	タグ付け	日付ベースのバージョンタグを付与	git tag -a vYYYYMMDD -m 'Release'
7	プッシュ	タグごとリモートへプッシュして完了	git push -u origin HEAD --tags

6 スキル定義 (.claude/skills/)

6-1. code-review スキル

ファイル	.claude/skills/code-review/SKILL.md
説明	変更差分を分析し、バグ・セキュリティ・品質の問題を指摘するコードレビュースキル
使用ツール	Read (ファイル参照) ・ Bash (git diff 実行) ・ Edit (修正提案の適用)
起動方法	ユーザーが「レビューして」などと指示したとき自動適用
出力形式	[HIGH] / [MEDIUM] / [LOW] のラベル付きで問題箇所と修正提案を提示

レビュー観点

観点	チェック内容
正確性	バグ・ロジックエラー・エッジケースの見落とし・型の不一致
セキュリティ	インジェクション攻撃・認証不備・機密情報の誤コミット

品質	変数/関数の命名・重複コード・不要な複雑性・可読性
テスト	テストカバレッジの不足・エッジケース漏れ・モックの妥当性

6-2. testing スキル

ファイル	.claude/skills/testing/SKILL.md
説明	テストコードを作成・実行するスキル。describe + it + AAA パターンを採用
使用ツール	Read・Bash・Edit・Write（テストファイルの新規作成も行う）
テスト構造	describe（テスト群）→ it（個別ケース）→ Arrange / Act / Assert の3段階
カバー範囲	正常系（Happy Path）・異常系（Error Path）・境界値（Edge Cases）

テストテンプレート（Python）

```
def test_機能名_条件_期待結果():  
  
    # Arrange（テストデータ・モックの準備）  
  
    input_value = create_test_fixture()  
  
    mock_service = MockService(return_value=expected)  
  
    # Act（テスト対象の処理を実行）  
  
    result = target_function(input_value, service=mock_service)  
  
    # Assert（結果を検証）  
  
    assert result == expected_value  
  
    assert mock_service.was_called_once()
```

7 スクリプト（scripts/）

7-1. auto-sync.ps1 — Windows 自動同期スクリプト

ファイル	scripts/auto-sync.ps1
目的	Windows PC 上の C:¥Claude フォルダを GitHub から自動的に最新状態に保つ
動作	ローカルとリモートのコミットハッシュを比較し、差分があれば git pull を実行
ログ出力	scripts/sync.log にタイムスタンプ付きでログを記録
対象ブランチ	master（固定）

```
# 動作フロー  
  
1. Set-Location C:¥Claude # リポジトリへ移動
```

2. `git fetch origin master` # リモートの最新情報を取得
3. ローカルHEAD と `origin/master` を比較
4. 同じ場合 → 「Already up to date.」をログに記録して終了
5. 異なる場合 → `git pull origin master` を実行してログに記録

7-2. setup-task-scheduler.ps1 — タスクスケジューラ登録スクリプト

ファイル	scripts/setup-task-scheduler.ps1
目的	auto-sync.ps1 を Windows タスクスケジューラに登録し、15分ごとに自動実行する
実行権限	管理者権限 (Run as Administrator) が必要
タスク名	ClaudeAutoSync
実行間隔	15 分ごと (\$interval 変数で変更可能)
実行条件	ネットワーク接続時のみ実行 / スタート遅延があっても実行
タイムアウト	最大 2 分 (-ExecutionTimeLimit)

8 ワークフロールール (CLAUDE.md)

8-1. 開発ワークフロー 6原則

ルール	内容と目的
Plan Mode を使う	調査フェーズと実装フェーズを明確に分離する。 いきなりコードを書かず、まず構造を把握してから実装に入る。
小さく実装・すぐ検証	小さく作る → 動作確認 → 改善 のサイクルを繰り返す。 一度に大きく変えず、確認できる単位で進める。
Todo リストで進捗管理	優先順位と進捗を常に可視化する。 作業前にタスクを列挙し、完了したものから消していく。
Commit は頻繁に	1 タスク完了ごとにすぐコミットする。複数タスクをまとめてコミットしない。
ブランチ戦略	main (本番) / feature/* (機能開発) / spike/* (実験) を目的に応じて使い分ける。
/compact を定期的に	コンテキストウィンドウが圧迫されたら /compact で圧縮して作業余白を確保する。 ※ 実行前に重要情報を CLAUDE.md へ退避すること。

8-2. コンテキスト管理ルール

CLAUDE.md を活用	重要な情報・ルール・約束事はすべて CLAUDE.md に記録する
ローカルコンテキスト	サブフォルダには .claude/ や CLAUDE.md を追加してスコープを分ける

ファイルサイズ制限	各ファイルは 200 行未満に保つ（可読性と管理のため）
/compact の注意	実行すると戻せないため、重要情報を先に CLAUDE.md へ退避してから実行

8-3. セキュリティポリシー

.env ファイル・認証情報	絶対にコミットしない。.gitignore と deny ルールで二重に防止。
外部副作用	DB 操作・外部 API 呼び出しは必ずユーザーの確認を取ってから実行する。
sudo コマンド	settings.json の deny ルールで強制禁止。原則として使用不可。

9 Git 情報

9-1. ブランチ構成

ブランチ名	種別	説明
master	本番ブランチ	安定した動作確認済みコードを格納
claude/quirky-newton-vLU0Z	開発ブランチ（ 現在）	現在の作業ブランチ。完了後 master へマージ

9-2. コミット履歴（全 9 件）

ハッシュ	ブランチ	コミットメッセージ
9b918db	claude/quirky- ...	Update report PDF with Japanese font (IPA Gothic) for proper rendering
a3e2c86	claude/quirky- ...	Add Claude Code environment report PDF and generation script
69ae13f	claude/quirky- ...	Fix PowerShell encoding: replace Japanese with English
b4fe020	claude/quirky- ...	Fix emoji encoding error in setup-task-scheduler.ps1
69ca2bd	claude/quirky- ...	Add Task Scheduler setup script for auto-sync
71cfe7b	master	Add auto-sync: Stop hook + Windows PowerShell sync script
e3cf942	master	Track .claude/ config files for GitHub-local sync
1323d56	master	Add CLAUDE.md with workspace rules and workflow guidelines
8559dad	master	Initial commit

9-3. .gitignore 設定

.claude/settings.local.json ← ローカル専用設定（個人の環境差異を除外）

.claude/hooks/ ← フックスクリプト（環境依存のため除外）

本レポートは Claude Code により自動生成されました | 2026-05-22 | ooaku-bit/claude